20. Documentation, APIs, and Semantic Versioning

20. Documentation, APIs, and Semantic Versioning

Phase: Phase 3 — Idioms and engineering judgment
Estimated time: 1 day
Slug: `20-docs-apis-semver`

Learning objectives

- Write Go doc comments that godoc renders
- Design stable exported APIs
- Apply semver and Go modules major versions
- Deprecate thoughtfully

Prerequisites

Complete prior numbered lessons. Type every example yourself.

Doc comments

```
// Package store implements durable persistence for users.
package store

// Find returns a user by ID.
// It returns ErrNotFound if no user exists.
func Find(ctx context.Context, id string) (*User, error)
```

Start with the name of the symbol. Run `go doc` locally.

Semver

- MAJOR: breaking changes (import path `/v2` for v2+)
- MINOR: compatible new features
- PATCH: bug fixes

API design

- Accept interfaces, return structs (often)
- Keep exported surface small
- Prefer options structs over endless parameters

Common mistakes (especially from Python instincts)

- Breaking APIs without a major version
- Empty doc comments that restate the name only poorly
- Exporting types “for later”

Exercises

Work in `exercises/20-docs-apis-semver/`. Read that folder’s `README.md`.

- **A (Drill):** make unit tests pass with the simplest correct code.
- **B (Apply):** extend with an edge case called out in the exercise README.
- **C (Challenge):** rewrite your solution from memory the next day without looking.

Do not open `solutions/20-docs-apis-semver/` until you have an honest failing attempt.

Self-check

1. What triggers a major version in Go modules?
2. How should a doc comment start?
3. Why keep the export surface small?

Further reading

- https://go.dev/doc/comment
- https://go.dev/doc/modules/release-workflow